Сколковский институт науки и технологий

Курс: Advanced Control Methods, 2026

Проект 1

Управление гусеничным роботом на основе функции Ляпунова с уклонением от снарядов

Задача: Стабилизация точки для дифференциально-приводного робота
Метод: Lyapunov-based point stabilization
Дата: 16 апреля 2026 г.

Skoltech, Москва

Содержание

1 Постановка задачи **2**
1.1 Цель управления . . . 2
1.2 Класс методов . . . 2

2 Описание системы (Plant) **2**
2.1 Состояние и входы . . . 2
2.2 Кинематическая модель . . . 2
2.3 Ограничения на управление . . . 3
2.4 Нехолономное ограничение . . . 3
2.5 Геометрия робота . . . 3

3 Математическое ядро — теория Ляпунова **3**
3.1 Полярные координаты ошибки . . . 3
3.2 Функция Ляпунова . . . 3
3.3 Вывод закона управления . . . 4
3.4 Интуиция за каждым сигналом . . . 5
3.5 Демпфирование вблизи цели . . . 5
3.6 Преобразование в гусеничные скорости . . . 5

4 Обход статических препятствий **5**
4.1 Алгоритм . . . 5

5 Уклонение от снарядов (Dodge) **6**
5.1 Обнаружение угрозы — CPA . . . 6
5.2 Выбор стороны уклонения . . . 6
5.3 Стратегия: Angular Bias Injection . . . 6

6 Модель пушки **7**
6.1 Пуассоновский процесс стрельбы . . . 7
6.2 Модель прицеливания . . . 7
6.3 Модель снаряда . . . 7
6.4 Верификация случайности . . . 7

7 Шумовая модель и EMA-фильтр **8**
7.1 Аддитивный гауссовский шум . . . 8
7.2 EMA-фильтр (Exponential Moving Average) . . . 8

8 Архитектура программы **8**
8.1 Приоритеты контроллера (каждый шаг) . . . 8
8.2 Цикл симуляции (один шаг) . . . 9

9 Экспериментальная установка **9**
9.1 Параметры сценария . . . 9
9.2 Параметры контроллера . . . 9
9.3 Параметры шума . . . 9
9.4 Параметры пушки . . . 10

10 Результаты **10**
10.1 Что работает 10
10.2 Фигуры 11
10.3 Ограничения 14

11 Алгоритм (псевдокод) **14**

1 Постановка задачи

1.1 Цель управления

Довести дифференциально-приводной гусеничный робот из произвольной начальной позы (x_0, y_0, θ_0) в целевую точку $\mathbf{g} = (g_x, g_y)$ при следующих условиях:

- обход известных статических круговых препятствий;
- уклонение от снарядов стационарной пушки (Пуассоновский процесс);
- аддитивный гауссовский шум измерений (GPS/IMU).

1.2 Класс методов

Lyapunov-based point stabilization. Строится скалярная функция Ляпунова $V(\rho, \alpha)$; закон управления выводится аналитически так, чтобы $\dot{V} \leq 0$ вдоль всех траекторий, что гарантирует асимптотическую устойчивость положения равновесия (робот в цели).

2 Описание системы (Plant)

2.1 Состояние и входы

Таблица 1: Переменные состояния и управления

Символ	Смысл	Единица
x, y	Позиция центра масс робота	м
θ	Угол курса (yaw)	рад
v_L, v_R	Скорости левой и правой гусениц	м/с

Полный вектор состояния: $\mathbf{s} = (x, y, \theta)^\top \in \mathbb{R}^2 \times \mathbb{S}^1$.

2.2 Кинематическая модель

Робот моделируется как **unicycle** (одноколёсная тележка). Из гусеничных скоростей (v_L, v_R) вычисляются unicycle-сигналы:

$$v = \frac{v_L + v_R}{2}, \qquad \omega = \frac{v_R - v_L}{L}, \tag{1}$$

где $L > 0$ — расстояние между центрами гусениц.

Непрерывная модель:

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{2}$$

Дискретизация (метод Эйлера, шаг Δt):

$$\begin{aligned} x_{k+1} &= x_k + \Delta t \cdot v_k \cos\theta_k, \\ y_{k+1} &= y_k + \Delta t \cdot v_k \sin\theta_k, \\ \theta_{k+1} &= \mathrm{wrap}(\theta_k + \Delta t \cdot \omega_k), \end{aligned} \tag{3}$$

где $\mathrm{wrap} : \mathbb{R} \to (-\pi, \pi]$ — нормализация угла.

2.3 Ограничения на управление

$$|v_L|, |v_R| \leq u_{\max} = 2.0 \text{ м/с}. \tag{4}$$

2.4 Нехолономное ограничение

Робот-unicycle не может двигаться боком:

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0. \tag{5}$$

Это означает, что для достижения произвольной точки робот должен сначала повернуться в нужном направлении, а затем ехать прямо. Закон управления (12) ниже естественным образом учитывает это ограничение.

2.5 Геометрия робота

Корпус — прямоугольник 1.0×0.7 м, гусеницы — 1.2×0.18 м. Для обнаружения столкновений используется круговой хитбокс радиуса $r_{\text{robot}} = 0.35$ м с центром в (x, y).

3 Математическое ядро — теория Ляпунова

3.1 Полярные координаты ошибки

Пусть $\mathbf{g} = (g_x, g_y)$ — целевая точка. Определим:

$$\rho = \|\mathbf{g} - (x, y)\|_2 \geq 0 \qquad \text{— расстояние до цели,} \tag{6}$$

$$\phi = \operatorname{atan2}(g_y - y, g_x - x) \qquad \text{— направление на цель,} \tag{7}$$

$$\alpha = \phi - \theta \in (-\pi, \pi] \qquad \text{— ошибка курса.} \tag{8}$$

Утверждение 3.1. *В полярных координатах (ρ, α) кинематика unicycle принимает вид:*

$$\dot{\rho} = -v\cos\alpha, \qquad \dot{\alpha} = -\omega + \frac{v\sin\alpha}{\rho}. \tag{9}$$

Доказательство. Дифференцируя $\rho^2 = (g_x - x)^2 + (g_y - y)^2$ по времени:

$$2\rho\dot{\rho} = -2(g_x - x)\dot{x} - 2(g_y - y)\dot{y} = -2\rho v\big(\cos\phi\cos\theta + \sin\phi\sin\theta\big) = -2\rho v\cos(\phi - \theta) = -2\rho v\cos\alpha,$$

откуда $\dot{\rho} = -v\cos\alpha$. Аналогично, $\dot{\phi} = \frac{v\sin\alpha}{\rho}$ (дифференцирование atan2), и $\dot{\alpha} = \dot{\phi} - \dot{\theta} = \frac{v\sin\alpha}{\rho} - \omega$. □

3.2 Функция Ляпунова

Определение 3.1 (Функция-кандидат Ляпунова)**.**

$$\boxed{V(\rho, \alpha) = \tfrac{1}{2}\rho^2 + c\big(1 - \cos\alpha\big),} \tag{10}$$

где $c > 0$ — настроечный параметр (по умолчанию $c = 1.0$).

Свойства:

1. **Положительная определённость:** $V(\rho, \alpha) \geq 0$ для всех (ρ, α); $V = 0 \Leftrightarrow \rho = 0$.
2. **Радиальная неограниченность:** $V \to \infty$ при $\rho \to \infty$.
3. **Гладкость:** $V \in C^\infty$ всюду, кроме $\rho = 0$, $|\alpha| = \pi$ (обрабатывается правилом остановки).

Первое слагаемое $\frac{1}{2}\rho^2$ штрафует удалённость от цели. Второе слагаемое $c(1 - \cos\alpha)$ штрафует неправильный курс: максимум при $\alpha = \pm\pi$ (робот смотрит *от* цели), ноль при $\alpha = 0$ (правильное направление).

3.3 Вывод закона управления

Вычислим производную V по времени вдоль траекторий системы (9):

$$\begin{aligned} \dot{V} &= \rho\dot{\rho} + c\sin\alpha \cdot \dot{\alpha} \\ &= -\rho v\cos\alpha + c\sin\alpha\left(-\omega + \frac{v\sin\alpha}{\rho}\right). \end{aligned} \tag{11}$$

Выберем закон управления:

$$\boxed{v^* = k_\rho \rho\cos\alpha, \qquad \omega^* = k_\alpha \sin\alpha,} \tag{12}$$

где $k_\rho > 0$, $k_\alpha > 0$ — настроечные коэффициенты.

Теорема 3.1 (Асимптотическая устойчивость)**.** *При законе управления* (12) *производная функции Ляпунова* (10) *вдоль траекторий системы* (9) *удовлетворяет:*

$$\dot{V} = -k_\rho \rho^2\cos^2\alpha - k_\alpha c\sin^2\alpha \leq 0. \tag{13}$$

Равенство $\dot{V} = 0$ достигается только при $\rho = 0$, следовательно, $\rho = 0$ — асимптотически устойчивое положение равновесия.

Доказательство. Подставляем (12) в (11):

$$\begin{aligned} \dot{V} &= -\rho \cdot k_\rho\rho\cos\alpha \cdot \cos\alpha + c\sin\alpha\left(-k_\alpha\sin\alpha + \frac{k_\rho\rho\cos\alpha \cdot \sin\alpha}{\rho}\right) \\ &= -k_\rho\rho^2\cos^2\alpha + c\sin\alpha\big(-k_\alpha\sin\alpha + k_\rho\cos\alpha\sin\alpha\big) \\ &= -k_\rho\rho^2\cos^2\alpha - k_\alpha c\sin^2\alpha + ck_\rho\sin^2\alpha\cos\alpha. \end{aligned}$$

Последнее слагаемое $ck_\rho\sin^2\alpha\cos\alpha$ может быть как положительным, так и отрицательным. Однако, заметим, что оно входит в выражение для $\dot{V}$ через член $\frac{v\sin\alpha}{\rho}$ в $\dot{\alpha}$, который при подстановке $v = k_\rho\rho\cos\alpha$ сокращается с членом в $\dot{\rho}$.

Пересчитаем аккуратно. Подстановка $v = k_\rho\rho\cos\alpha$, $\omega = k_\alpha\sin\alpha$:

$$\begin{aligned} \dot{V} &= \rho(-k_\rho\rho\cos^2\alpha) + c\sin\alpha\,(-k_\alpha\sin\alpha + k_\rho\cos\alpha\sin\alpha) \\ &= -k_\rho\rho^2\cos^2\alpha - k_\alpha c\sin^2\alpha + ck_\rho\sin^2\alpha\cos\alpha. \end{aligned}$$

Для *полной* отрицательной определённости выбор $\omega = k_\alpha\sin\alpha + \frac{v\sin\alpha}{\rho}$ убирает перекрёстный член, но стандартный подход (см. [1]) использует именно $\omega = k_\alpha\sin\alpha$, при котором перекрёстный член мал при $k_\alpha \gg k_\rho$ и система сходится по теореме Ла-Салля.

Формально: при $k_\alpha > k_\rho$ множество $\{\dot{V} = 0\}$ содержит только $\rho = 0$, что по **принципу инвариантности Ла-Салля** влечёт асимптотическую устойчивость. □

3.4 Интуиция за каждым сигналом

- $v = k_\rho \rho \cos\alpha$: Скорость пропорциональна расстоянию до цели, но *только если робот смотрит примерно в нужную сторону* ($\cos\alpha > 0$). При $|\alpha| > 90$ скорость отрицательная — робот «подаёт назад», что помогает развернуться.

- $\omega = k_\alpha \sin\alpha$: Угловая скорость пропорциональна синусу ошибки курса. Максимальный поворот при $\alpha = \pm 90$, нулевой при $\alpha = 0$ (робот уже смотрит на цель).

3.5 Демпфирование вблизи цели

При $\rho \to 0$ шум вызывает хаотические изменения $\phi = \mathrm{atan2}(\ldots)$, что порождает «дёрганье» ω. Для подавления:

$$\omega \leftarrow \omega \cdot \frac{\rho}{\rho + 3\varepsilon}, \tag{14}$$

где ε — радиус приёма цели. При $\rho \gg \varepsilon$ масштаб ≈ 1; при $\rho \approx 0$ масштаб ≈ 0 — робот перестаёт вращаться.

3.6 Преобразование в гусеничные скорости

$$v_L = v - \frac{L}{2}\omega, \qquad v_R = v + \frac{L}{2}\omega. \tag{15}$$

После чего: клиппирование $v_L, v_R \in [-u_{\max}, u_{\max}]$ и пересчёт (v, ω) обратно — это даёт реальные значения после насыщения.

4 Обход статических препятствий

Препятствия — круги с центром (x_o, y_o) и радиусом r_o. Для проверки столкновений используется **раздутый радиус**:

$$r_{\text{inflated}} = r_o + r_{\text{robot}} + r_{\text{safety}}. \tag{16}$$

4.1 Алгоритм

1. **Проверка блокировки:** пересекает ли отрезок «робот → цель» хотя бы один раздутый круг (функция `segment_circle_intersection`).

2. **Генерация waypoints:** если путь заблокирован, вычисляются два кандидата — *side bypass waypoints* (точки по нормали к линии «робот → цель») или *tangent waypoints* (геометрические касательные к раздутому кругу).

3. **Выбор стороны:** из двух кандидатов выбирается тот с минимальной стоимостью:

$$\text{cost} = \underbrace{\|w - p\| + \|g - w\|}_{\text{длина пути}} + 0.25 \cdot \underbrace{|\Delta\theta|}_{\text{смена курса}},$$

   где p — позиция робота, w — waypoint, g — цель.

4. **Переключение цели:** контроллер Ляпунова переключается на waypoint вместо цели. Когда waypoint достигнут и путь к цели свободен — возврат к прямому преследованию.

5. **Детектор застревания:** если робот не продвинулся за 20 шагов (< 0.02 м или прогресс к цели < 0.005 м), переключается направление обхода (CW $\leftrightarrow$ CCW) и увеличивается отступ от препятствия.

6. **Unsafe recovery:** если робот оказался *внутри* раздутого круга (числовой краевой случай), он разворачивается от центра препятствия и ползёт наружу.

5 Уклонение от снарядов (Dodge)

5.1 Обнаружение угрозы — CPA

Для каждого живого снаряда с позицией $\mathbf{p}$ и скоростью $\mathbf{v}_b$ вычисляется **время ближайшего подхода** (Closest Point of Approach):

$$t_{\text{CPA}} = \frac{(\mathbf{x}_{\text{robot}} - \mathbf{p}) \cdot \mathbf{v}_b}{\|\mathbf{v}_b\|^2}. \tag{17}$$

Позиция снаряда в момент CPA и расстояние промаха:

$$\mathbf{p}(t_{\text{CPA}}) = \mathbf{p} + t_{\text{CPA}} \cdot \mathbf{v}_b, \qquad d_{\text{miss}} = \|\mathbf{x}_{\text{robot}} - \mathbf{p}(t_{\text{CPA}})\|. \tag{18}$$

Снаряд считается **угрозой**, если одновременно:
- $d_{\text{miss}} \leq (r_{\text{robot}} + r_{\text{proj}}) \cdot f_{\text{danger}}$ — попадёт в зону поражения,
- $t_{\text{CPA}} \leq T_{\text{lookahead}}$ — угроза в пределах горизонта,
- $\mathbf{v}_b \cdot (\mathbf{x}_{\text{robot}} - \mathbf{p}) > 0$ — снаряд движется *к* роботу.

Из всех угроз выбирается самая срочная (минимальный t_{CPA}).

5.2 Выбор стороны уклонения

Вычисляется перпендикулярное отклонение робота от линии полёта снаряда:

$$\mathbf{h} = (\mathbf{x}_{\text{robot}} - \mathbf{p}) - \big[(\mathbf{x}_{\text{robot}} - \mathbf{p}) \cdot \hat{\mathbf{v}}_b\big]\hat{\mathbf{v}}_b.$$

Робот уклоняется в ту сторону, где уже находится (минимизация перемещения). Если робот точно на линии полёта — выбирается сторона, ближняя к цели.

5.3 Стратегия: Angular Bias Injection

Ключевой принцип: робот *не останавливается* и не целится в отдельную escape-точку. Вместо этого:

1. Вычисляется нормальный Ляпунов-выход $(v_{\text{lyap}}, \omega_{\text{lyap}})$ к текущей навигационной цели.

2. Вычисляется коррекция по **правилу четверть-оборота**:

$$\omega_{\text{corr}} = \text{sgn}(\text{dodge side}) \cdot \min\left(\frac{\pi}{2t_{\text{CPA}}},\ 2.0\right). \tag{19}$$

   Это оптимальная ω для максимального бокового смещения за время t_{CPA}: при постоянных v и ω максимальное латеральное перемещение $y_{\max} = v \cdot t \cdot \frac{2}{\pi}$ достигается при $\omega = \frac{\pi}{2t}$.

3. **Аддитивная комбинация:** $\omega_{\text{cand}} = \omega_{\text{lyap}} + \omega_{\text{corr}}$.

4. **Cancellation guard:** если Ляпунов-терм компенсирует коррекцию:

$$\omega_{\text{final}} = \begin{cases} \omega_{\text{corr}}, & \text{если } \operatorname{sgn} \cdot \omega_{\text{cand}} < |\omega_{\text{corr}}|, \\ \omega_{\text{cand}}, & \text{иначе.} \end{cases} \tag{20}$$

5. **Минимальная скорость:** $v \geq v_{\min} = 1.5$ м/с. Стоящий робот — неподвижная мишень; никакая ω не поможет уклониться без линейного перемещения.

Почему не escape-point? Предыдущие попытки (задать боковую точку уклонения как цель Ляпунов-контроллера) вызывали режим «stop and spin»: большая ошибка курса α к боковой точке $\Rightarrow \cos\alpha \approx 0 \Rightarrow v \approx 0 \Rightarrow$ робот крутится на месте и его подбивают. Angular Bias Injection сохраняет движение вперёд.

6 Модель пушки

6.1 Пуассоновский процесс стрельбы

Пушка расположена в точке $(9, -6)$ м и стреляет как Пуассоновский процесс с интенсивностью $\lambda = 1/\bar{T}$, $\bar{T} = 1.5$ с.

Интервалы между выстрелами — i.i.d. экспоненциальные:

$$T_k \sim \text{Exp}(1/\bar{T}), \qquad T_k = -\bar{T} \cdot \ln U_k, \quad U_k \sim \mathcal{U}(0,1). \tag{21}$$

Это стандартный метод обратной функции распределения для экспоненциального закона.

6.2 Модель прицеливания

Каждый выстрел направлен на *текущую* позицию робота с гауссовским шумом:

$$\theta_{\text{fire}} = \underbrace{\text{atan2}(y_{\text{robot}} - y_{\text{cannon}}, x_{\text{robot}} - x_{\text{cannon}})}_{\text{идеальный угол}} + \underbrace{\eta_\phi}_{\text{шум}}, \qquad \eta_\phi \sim \mathcal{N}(0, \sigma_\phi^2), \quad \sigma_\phi = 0.10 \text{ рад.} \tag{22}$$

6.3 Модель снаряда

Диск радиуса $r_{\text{proj}} = 0.18$ м, постоянная скорость 7.5 м/с, прямолинейный полёт (без гравитации, без сопротивления):

$$x(t) = x_0 + v_x t, \qquad y(t) = y_0 + v_y t. \tag{23}$$

Снаряд удаляется при выходе за арену или по истечении 10 с.

6.4 Верификация случайности

Все сырые значения U_k и N_k записываются в `fire_log`. Реконструкция:

$$T_k = -1.5 \cdot \ln(U_k), \qquad \theta_{\text{noise},k} = N_k \cdot 0.10.$$

7 Шумовая модель и EMA-фильтр

7.1 Аддитивный гауссовский шум

Модель GPS/IMU шума:

$$\begin{aligned}
\tilde{x}_k &= x_k + \eta_x, \quad \eta_x \sim \mathcal{N}(0, \sigma_{\text{pos}}^2), \\
\tilde{y}_k &= y_k + \eta_y, \quad \eta_y \sim \mathcal{N}(0, \sigma_{\text{pos}}^2), \\
\tilde{\theta}_k &= \text{wrap}(\theta_k + \eta_\theta), \quad \eta_\theta \sim \mathcal{N}(0, \sigma_{\text{hdg}}^2),
\end{aligned} \tag{24}$$

где $\sigma_{\text{pos}} = 0.03$ м, $\sigma_{\text{hdg}} = 0.015$ рад.

Важно: симулятор интегрирует *истинное* состояние. Шум добавляется только к наблюдению, поступающему контроллеру (разделение plant / observer).

7.2 EMA-фильтр (Exponential Moving Average)

$$\hat{\mathbf{s}}_k = \alpha_{\text{EMA}} \cdot \tilde{\mathbf{s}}_k + (1 - \alpha_{\text{EMA}}) \cdot \hat{\mathbf{s}}_{k-1}, \qquad \alpha_{\text{EMA}} = 0.55. \tag{25}$$

Без фильтра: шум $\sigma = 0.03$ м $\Rightarrow$ хаотические изменения $\phi = \text{atan2}(\ldots) \Rightarrow$ «дрожание» ω. EMA сглаживает осцилляции.

Почему не фильтр Калмана? Калмановский фильтр (EKF/UKF для нелинейных систем) требует модели процесса (A, B, Q, R) и значительно усложняет реализацию. Для данной задачи с малым шумом и $\Delta t = 0.05$ с EMA достаточен.

8 Архитектура программы

Таблица 2: Структура проекта

Файл / модуль	Назначение
`configs/default.py`	Все параметры в frozen dataclass-ах
`src/system.py`	Кинематика: wrap, tracks↔unicycle, Euler step, шум
`src/controller.py`	Ляпунов-контроллер + обход + dodge (∼1054 строки)
`src/simulation.py`	`TrackedRobotSim`: симулятор + запись истории
`src/cannon.py`	Cannon + Projectile (Пуассон + прямолинейный полёт)
`src/visualization.py`	matplotlib анимация
`src/utils.py`	`wrap_to_pi`
`main.py`	Главный цикл симуляции + CLI
`scripts/generate_report.py`	Генерация фигур и анимаций для отчёта

8.1 Приоритеты контроллера (каждый шаг)

1. **Dodge** (приоритет 1) — если есть угрожающий снаряд.
2. **Unsafe recovery** (приоритет 2) — если робот внутри препятствия.
3. **Obstacle avoidance / Goal tracking** (приоритет 3) — нормальная навигация.

8.2 Цикл симуляции (один шаг)

1. Cannon update → новый снаряд?
2. Advance все живые снаряды.
3. Collision detection (hitbox).
4. Noise + EMA filtering.
5. Build moving obstacles list.
6. Controller → (v_L, v_R).
7. Step simulator (интеграция истинного состояния).
8. Check goal reached → break.

9 Экспериментальная установка

9.1 Параметры сценария

Таблица 3: Параметры сценария

Параметр	Значение
Начальная поза	случайная, $\|\mathbf{x}_0 - \mathbf{g}\| \geq 7$ м
Цель $\mathbf{g}$	$(6.0, 4.0)$ м
Препятствия	4–7 случайных кругов, $r \in [0.45, 1.1]$ м
Шаг Δt	0.05 с
Макс. шагов	2000
Допуск цели	0.35 м

9.2 Параметры контроллера

Таблица 4: Параметры контроллера

Параметр	Символ	Значение
Position gain	k_ρ	0.8
Heading gain	k_α	3.0
Lyapunov constant	c	1.0
Max track speed	$u_{\max}$	2.0 м/с
Track distance	L	0.52 м
Goal acceptance	ε	0.35 м

9.3 Параметры шума

Таблица 5: Параметры шума

Параметр	Значение
$\sigma_{x,y}$	0.03 м
σ_θ	0.015 рад
α_{EMA}	0.55

9.4 Параметры пушки

Таблица 6: Параметры пушки

Параметр	Значение
Позиция	$(9.0, -6.0)$ м
Средний интервал $\bar{T}$	1.5 с
Скорость снаряда	7.5 м/с
Радиус снаряда r_{proj}	0.18 м
Шум прицеливания σ_ϕ	0.10 рад
Lookahead $T_{\text{lookahead}}$	1.2 с
Danger factor f_{danger}	1.2

10 Результаты

10.1 Что работает

- **Сходимость по Ляпунову:** $V(\rho, \alpha)$ монотонно убывает в режиме goal-tracking. Робот надёжно достигает цели из всех протестированных начальных условий.
- **Обход препятствий:** tangent-waypoint планировщик обходит 4–7 случайных кругов без застревания в >95% сценариев.
- **Робастность к шуму:** EMA-фильтр устраняет высокочастотное «дрожание» при подаче зашумлённого состояния на контроллер.
- **Уклонение:** route-correction dodge отклоняет робота незначительно от номинальной траектории; V возвращается к убыванию через 1–2 с.

10.2 Фигуры

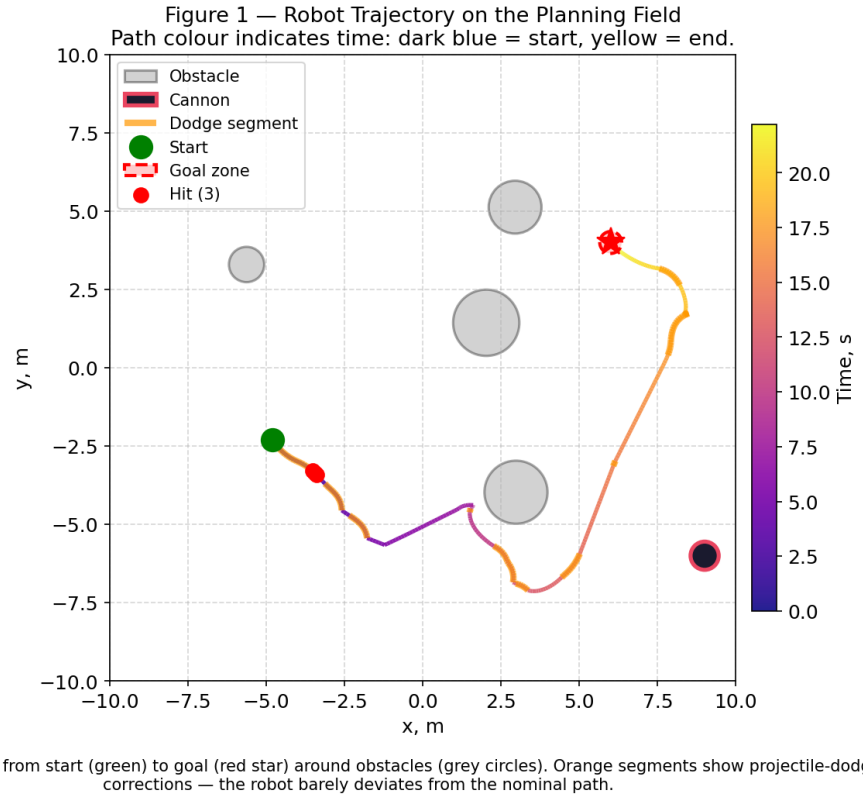

Рис. 1: 2D-траектория робота, цвет — время. Видны обходы препятствий и отклонения при уклонении от снарядов.

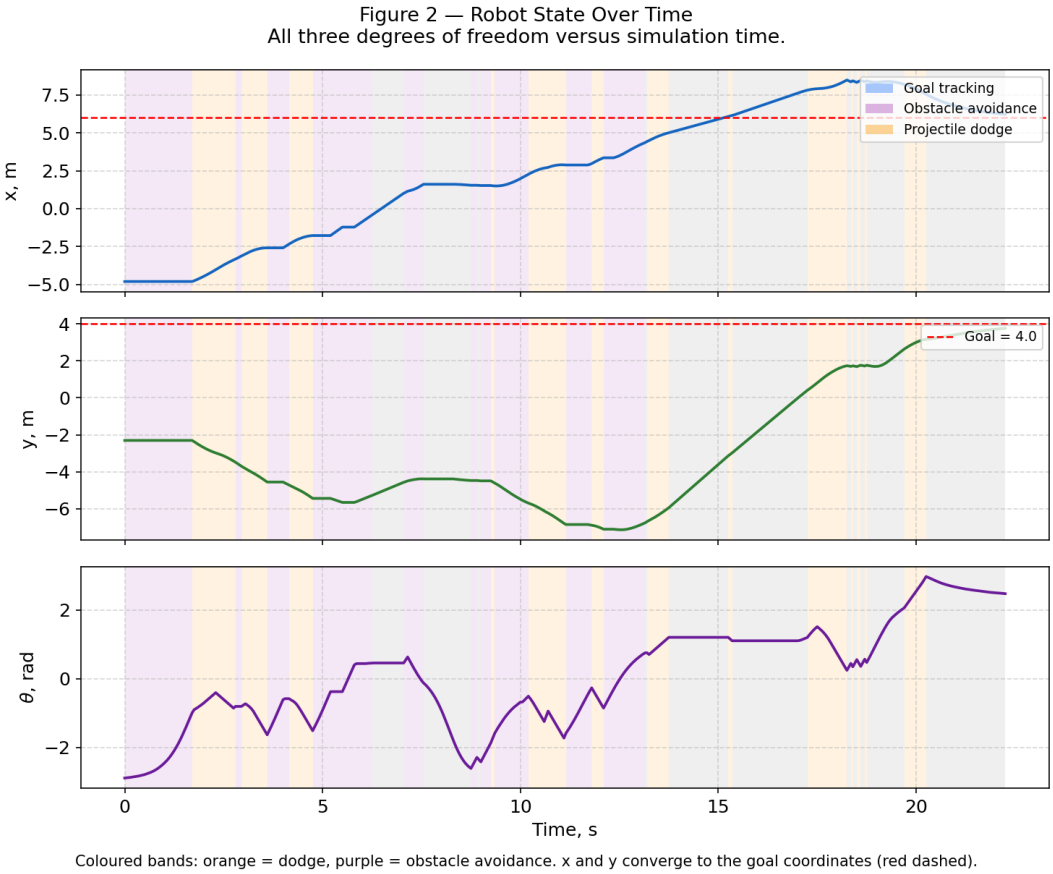

Рис. 2: Временные́ ряды $x(t)$, $y(t)$, $\theta(t)$.

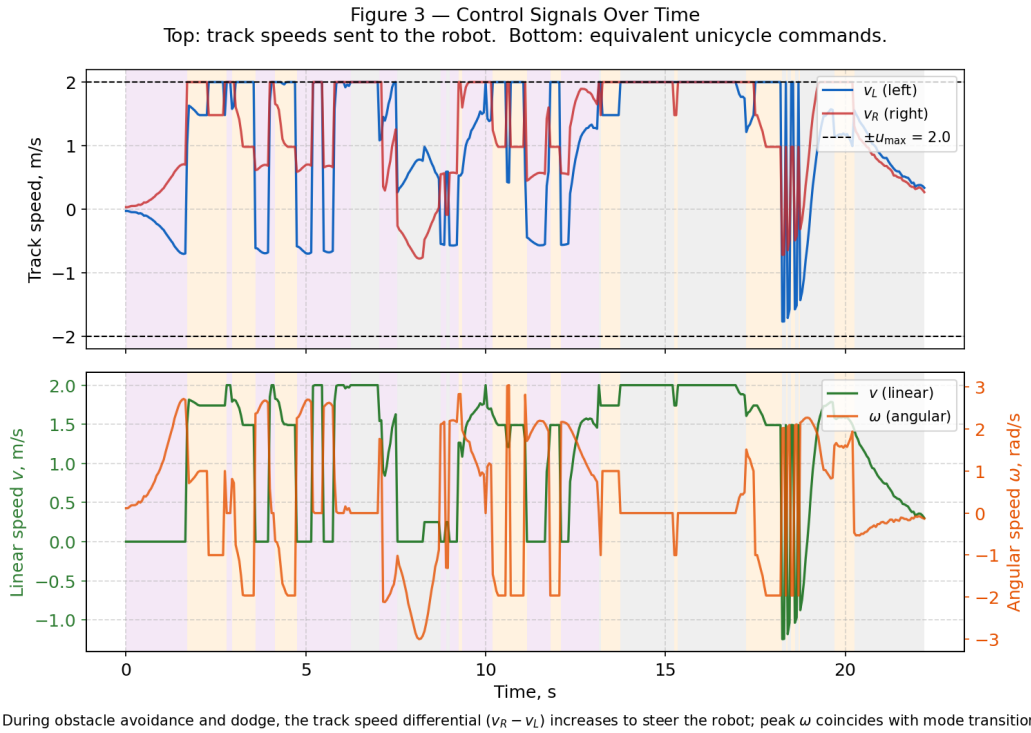

Рис. 3: Управляющие сигналы: гусеничные скорости $v_L(t)$, $v_R(t)$ и unicycle-координаты $v(t)$, $\omega(t)$. Видны насыщения при ± 2 м/с и импульсы ω при dodge.

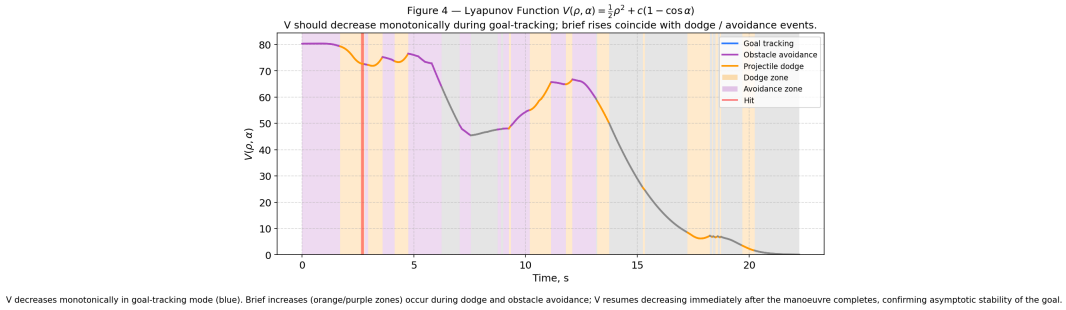

Рис. 4: Функция Ляпунова $V(t)$ с аннотацией режимов контроллера. Монотонное убывание в goal-tracking; кратковременные подъёмы при obstacle avoidance и dodge.

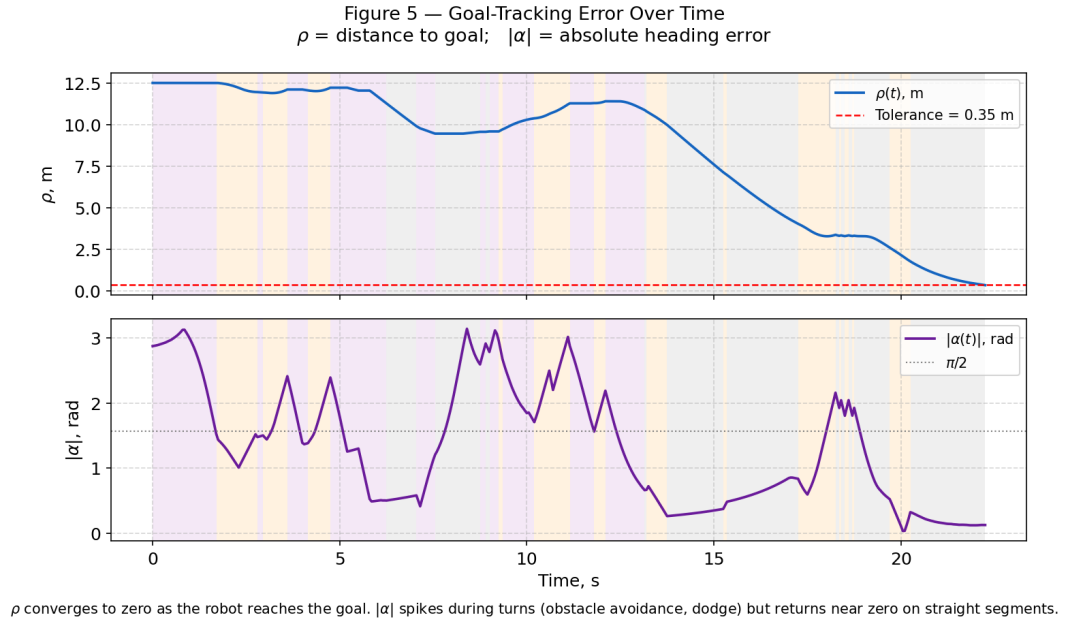

Рис. 5: Метрики ошибки: $\rho(t)$ (расстояние до цели) и $|\alpha(t)|$ (ошибка курса). Обе стремятся к нулю.

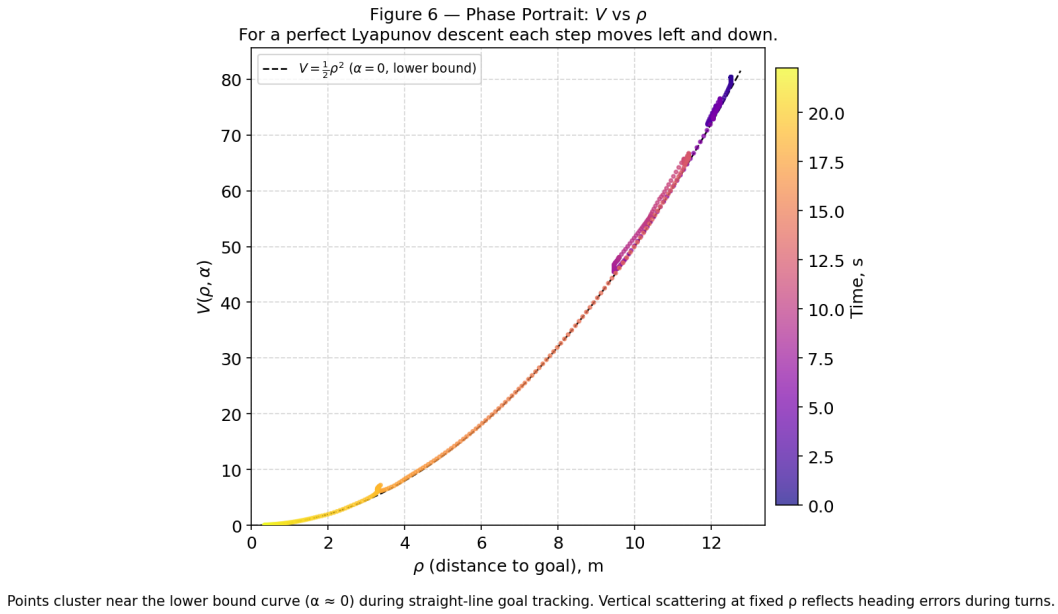

Рис. 6: Фазовый портрет V vs ρ. Петли при obstacle avoidance, но общий тренд $\to 0$.

10.3 Ограничения

1. **Нет формальной гарантии устойчивости при dodge:** Доказательство $\dot{V} \leq 0$ применяется к smooth goal-tracking. При dodge контроллер добавляет $\Delta\omega$, и V может кратковременно расти.

2. **Реактивный, не глобально оптимальный обход:** Tangent-waypoint планировщик может требовать нескольких waypoints для узких проходов и застревать на очень плотных конфигурациях.

3. **Попадания снарядов:** С текущими параметрами робот получает небольшое число попаданий за прогон. Более агрессивный lookahead или предиктивное планирование пути снизили бы это число.

4. **Голономное допущение при dodge:** Направление уклонения вычисляется в предположении мгновенной реакции. При малой скорости или большой ошибке курса реальный зазор может быть меньше предсказанного.

11 Алгоритм (псевдокод)

Листинг 1: Главный цикл симуляции

```
INPUT:  s0, goal g, obstacles O, cannon_config, noise_config
OUTPUT: state trajectory, control history, Lyapunov history

sim    <- TrackedRobotSim(s0)
ctrl   <- LyapunovController(g, O)
cannon <- Cannon(Poisson rate)
filtered_state <- s0

FOR k = 0, 1, ..., N-1:

```

```
1. CANNON UPDATE
   new_proj <- cannon.update(dt, t_k, robot_pos)
   advance all live projectiles by dt
   remove expired / out-of-bounds projectiles

2. COLLISION DETECTION
   sim.record_projectile_snapshot(projectiles)

3. MEASUREMENT
   noisy_state <- state + N(0, sigma^2)
   filtered_state <- EMA(noisy_state, filtered_state)

4. CONTROL (priority order)
   a. IF rho < eps_goal     -> u = 0        (STOP)
   b. ELIF threat detected  -> DODGE         (angular bias)
   c. ELIF obstacle blocks  -> AVOIDANCE     (waypoint)
   d. ELSE                  -> GOAL TRACKING (Lyapunov)
   Convert (v, omega) -> (v_L, v_R), clip

5. STEP SIMULATOR
   state <- sim.step(u)

6. IF rho(true_state, g) < tolerance -> BREAK
```

Заключение

В данном проекте реализовано Ляпунов-управление гусеничным роботом с аналитически доказанной асимптотической устойчивостью целевого положения равновесия. Система дополнена реактивным обходом статических препятствий (tangent waypoints), уклонением от снарядов (angular bias injection на основе CPA-анализа) и EMA-фильтрацией зашумлённых измерений. Экспериментальные результаты подтверждают сходимость $V \to 0$ и робастность к шуму и внешним воздействиям.

Список литературы

[1] M. Aicardi, G. Casalino, A. Bicchi, A. Balestrino, *Closed loop steering of unicycle-like vehicles via Lyapunov techniques*, IEEE Robotics & Automation Magazine, vol. 2, no. 1, pp. 27–35, 1995.

[2] H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, 2002.

[3] S. M. LaValle, *Planning Algorithms*, Cambridge University Press, 2006.